CS7CS2 – Optimisation for Machine Learning
Student Name: Stephen Komolafe
Student Number: 21336975

Week 6 Assignment

Q1:

Synthetic data for the linear regression model:

$$\hat{y} = X\theta, \qquad \theta \in \mathbb{R}^2$$

was generated in accordance with the assignment's specifications. A dataset of size $m = 1000$ was created where each element (i, j) of the design matrix X was sampled from a standard normal distribution $N(0, 1)$. The true parameter vector was set to $\theta^* = [3, 4]^T$, and the response variable y was generated using $y = X\theta^* + \epsilon$, where the noise term $\epsilon \sim N(0, \sigma^2 I)$, with $\sigma = 1$, hence $\epsilon \sim N(0, I)$.

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(0)

m = 1000
d = 2
sigma = 1

theta_star = np.array([3,4])
X = np.random.randn(m,d)
epsilon = sigma*np.random.randn(m)

y = X @ theta_star + epsilon

theta0 = np.array([1.,1.])
```

The loss function used was the mean squared error loss:

$$L(\theta) = \left(\frac{1}{2m}\right) \|X\theta - y\|_2^2.$$

The parameter vector was initialised at $\theta_0 = [1, 1]^T$. In the implementation, separate Python functions were defined for the loss and its gradient:

$$\nabla L(\theta) = \frac{1}{2m} 2X^T(X\theta - y)$$

$$= \frac{1}{m} X^T(X\theta - y)$$

using NumPy operations. These functions were then used by the optimisation algorithms implemented in the following parts.

```
def loss(theta):
    return (1/(2*m))*np.linalg.norm(X@theta - y)**2

def grad(theta):
    return (1/m)*X.T@(X@theta - y)
```

(a) Full-batch Gradient Descent

As a baseline method, full-batch gradient descent (GD) was implemented. At each iteration the gradient:

$$\nabla L(\theta) = \frac{1}{m} X^T (X\theta - y),$$

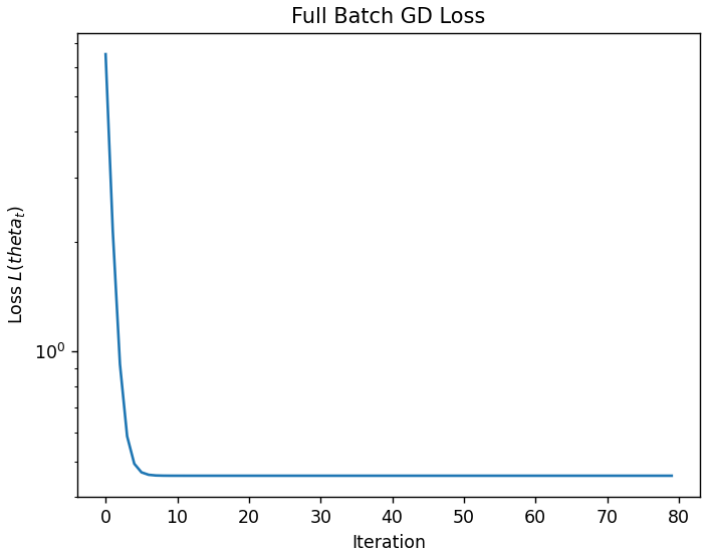
was computed using the entire dataset, and the parameters were updated according to:

$$\theta_{t+1} = \theta_t - \alpha \nabla L(\theta_t),$$

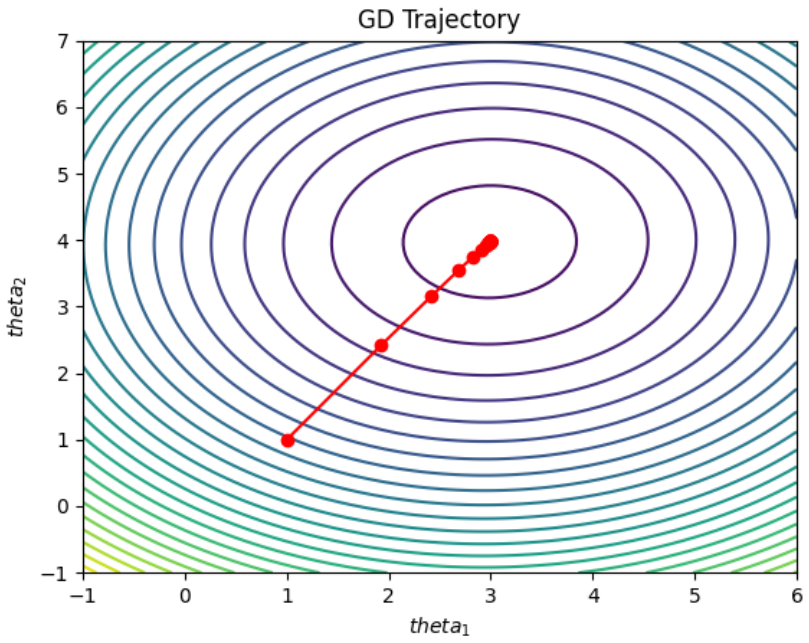
with a constant step size $\alpha = 0.5$. The algorithm was run for 80 iterations starting from the initial parameter vector.

```
alpha = 0.5
T = 80
theta = theta0.copy()

losses_gd = []
traj_gd = [theta.copy()]

for t in range(T):
    losses_gd.append(loss(theta))
    theta = theta - alpha*grad(theta)
    traj_gd.append(theta.copy())

traj_gd = np.array(traj_gd)
```

During optimisation, the value of the loss (`losses_gd`) and trajectory (`traj_gd`) at each iteration were stored to allow visualisation of the optimisation process.



The first plot shows the full-batch GD loss $L(\theta_t)$ as a function of iteration on a logarithmic scale. The loss decreases rapidly during the initial iterations, dropping from approximately 6.53 to below 0.50 within only a few steps, before gradually stabilising around 0.456. This behaviour is typical for gradient descent, where large improvements occur early in optimisation followed by slower refinement as the parameters approach the minimum.



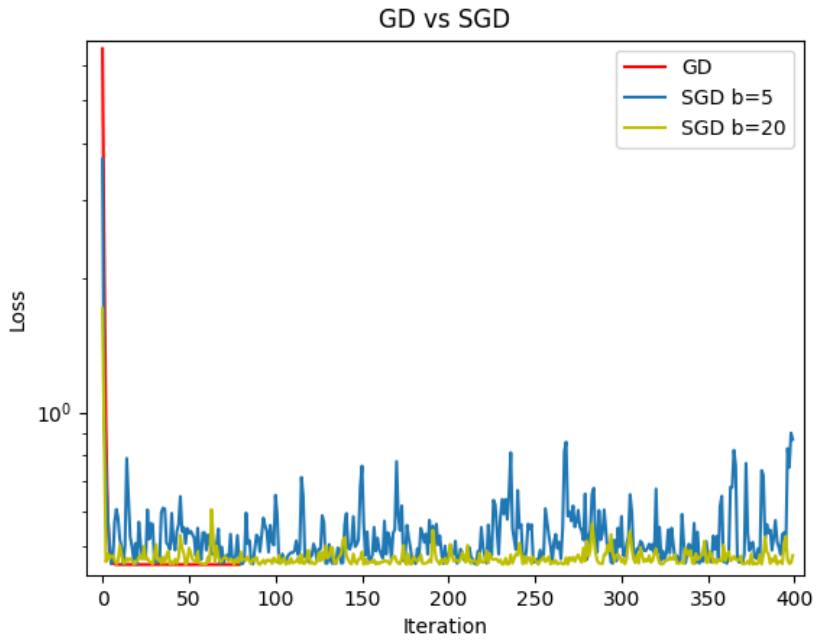
The second plot visualises the optimisation trajectory on a contour plot of the loss function $L(\theta_1, \theta_2)$. The contour surface was generated by evaluating the loss over a meshgrid of parameter values, and the successive parameter updates from gradient descent were overlaid to illustrate how the algorithm moves through the parameter space toward the optimum. The algorithm converges to an estimate of approximately $\theta = (2.99, 3.97)$, which is sufficiently close to the true parameter $(3, 4)$, with a parameter error of ≈ 0.027. The trajectory therefore shows a smooth path toward the minimum of the quadratic loss surface.

(b) Mini-batch Stochastic Gradient Descent

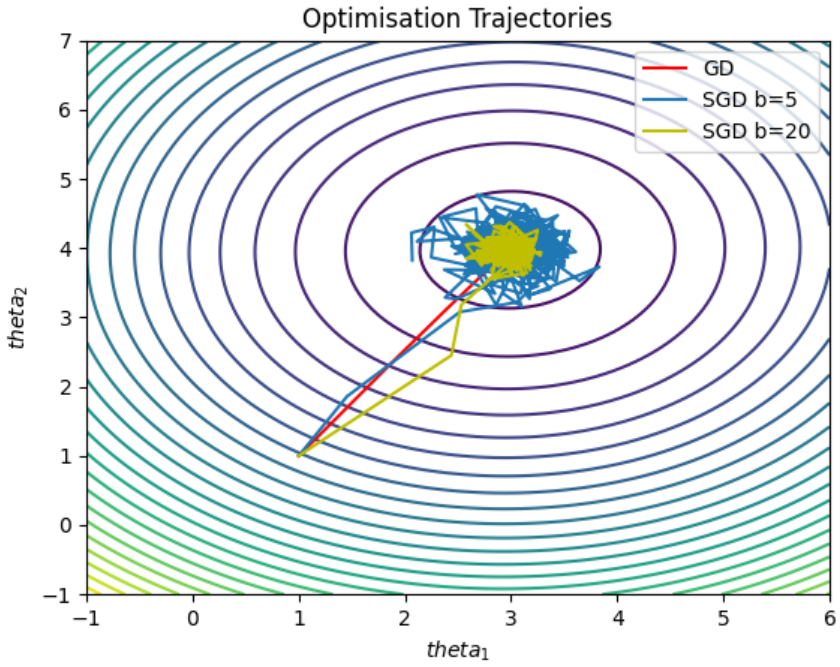
Mini-batch stochastic gradient descent (SGD) was implemented as a function to compare the effect of batch size on optimisation behaviour. Instead of computing the gradient using the full dataset, gradients were estimated using small randomly sampled subsets of the data. The dataset indices were shuffled once per epoch to ensure randomness in the mini-batch selection.

```
def SGD(batch_size, updates, alpha):
    theta = theta0.copy()
    traj = [theta.copy()]
    losses = []
    idx = np.arange(m)

    for t in range(updates):
        if t % (m//batch_size) == 0:
            np.random.shuffle(idx)

        batch = idx[(t*batch_size)%m : (t*batch_size)%m + batch_size]
        Xb = X[batch]
        yb = y[batch]
        g = (1/batch_size)*Xb.T@(Xb@theta - yb)
        theta = theta - alpha*g
        losses.append(loss(theta))
        traj.append(theta.copy())

    return np.array(losses), np.array(traj)
```

The update rule remained the same as in gradient descent but used a batch gradient computed from a subset of size b. The algorithm was run for 400 updates with the same learning rate $\alpha = 0.5$. Two batch sizes were compared: $b = 5$ and $b = 20$. The loss values and trajectory of the SGD for each batch size were stored and plotted against the number of iterations.



The first plot compares the loss curves for full-batch GD and the two SGD variants. While all methods reduce the objective value, the SGD curves exhibit noticeable fluctuations due to the stochastic nature of the gradient estimates. The magnitude of these fluctuations depends on the batch size. In particular, the smaller batch size $b = 5$ produces significantly noisier updates and converges to a higher final loss (≈ 0.869) compared with $b = 20$ (≈ 0.478) and full-batch GD (≈ 0.456). This shows how larger batches produce more accurate gradient estimates and therefore more stable convergence.



The second plot shows the optimisation trajectories of GD, SGD with $b = 5$, and with $b = 20$ on the same contour plot of the loss function. The GD trajectory follows a smooth path toward the minimum, reflecting the deterministic gradient updates. In contrast, the SGD trajectories appear noisier, especially for $b = 5$, where the updates fluctuate substantially around the optimal region. The larger batch size $b = 20$ produces a trajectory that is noticeably smoother and closer to the GD path, reflecting the reduced variance in the gradient estimate.

(c) Effect of SGD Noise: NAG and Adagrad

To further investigate stochastic optimisation behaviour, two adaptive optimisation methods were implemented and compared with standard SGD: Nesterov Accelerated Gradient (NAG) and Adagrad. Both methods were implemented using a batch size of b=10 and 400 updates.

```
def NAG(batch_size,updates,alpha,beta=0.9):
    theta = theta0.copy()
    v = np.zeros_like(theta)
    traj=[theta.copy()]
    losses=[]
    idx=np.arange(m)

    for t in range(updates):
        if t%(m//batch_size)==0:
            np.random.shuffle(idx)

        batch=idx[(t*batch_size)%m:(t*batch_size)%m+batch_size]
        Xb=X[batch]
        yb=y[batch]


        lookahead = theta - beta*v
        g=(1/batch_size)*Xb.T@(Xb@lookahead - yb)
        v = beta*v + alpha*g
        theta = theta - v

        losses.append(loss(theta))
        traj.append(theta.copy())

    return np.array(losses), np.array(traj)
```

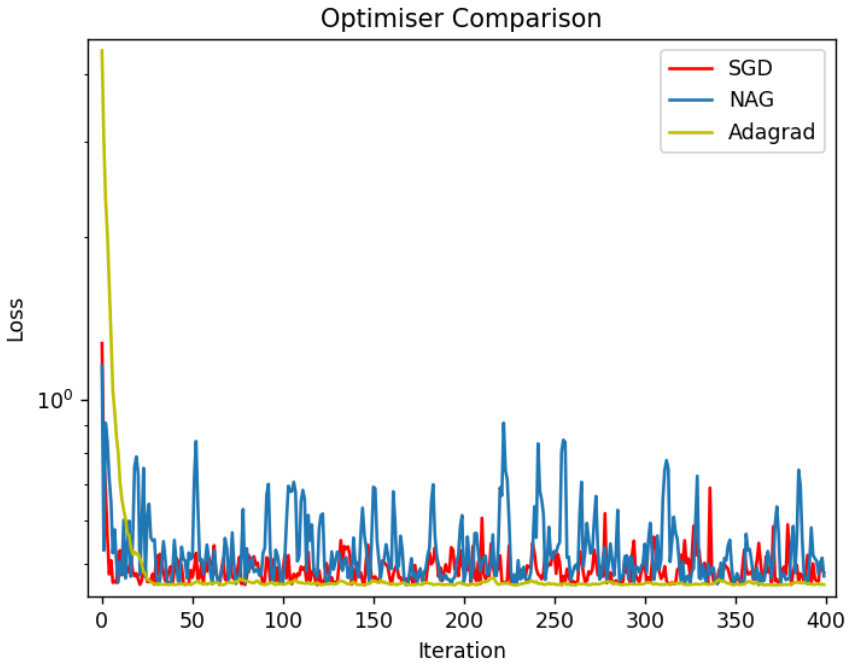
NAG introduces a momentum term that accelerates convergence by incorporating information from previous updates. In the implementation, a velocity vector was maintained and updated using a momentum coefficient β=0.9, with the gradient evaluated at a `lookahead` point.

```
def Adagrad(batch_size,updates,alpha,eps=1e-8):
    theta = theta0.copy()
    G = np.zeros_like(theta)
    traj=[theta.copy()]
    losses=[]
    idx=np.arange(m)
```

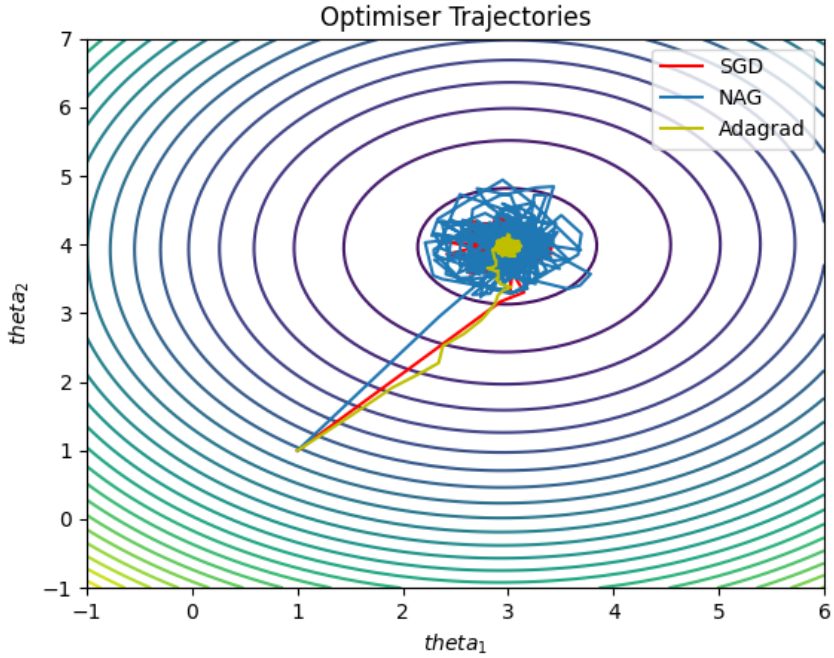
```
    for t in range(updates):
        if t%(m//batch_size)==0:
            np.random.shuffle(idx)

        batch=idx[(t*batch_size)%m:(t*batch_size)%m+batch_size]
        Xb=X[batch]
        yb=y[batch]

        g=(1/batch_size)*Xb.T@(Xb@theta - yb)
        G += g**2
        theta = theta - alpha/(np.sqrt(G)+eps)*g

        losses.append(loss(theta))
        traj.append(theta.copy())

    return np.array(losses), np.array(traj)
```

Adagrad, in contrast, adapts the learning rate for each parameter individually by accumulating the squared gradients over time and scaling updates accordingly.

The loss values (`loss_nag`, `loss_ada`) and trajectory (`traj_nag`, `traj_ada`) of the optimisation methods were stored and plotted against the number of iterations.



The first plot compares the loss curves for constant-step SGD, NAG, and Adagrad. All three methods successfully reduce the objective function, but they exhibit slightly different convergence behaviour. Adagrad achieves the lowest final loss (≈ 0.457), which is close to the full-batch GD result. NAG also improves upon standard SGD, reaching a loss of ≈ 0.474, while SGD converges to a slightly higher value (≈ 0.481). This demonstrates the benefit of adaptive or momentum-based optimisation methods.



The second plot shows the trajectories of the three methods on the contour plot of the loss function. While all trajectories move toward the optimal region near $(3, 4)$, the paths differ due to the update rules of the algorithms. The SGD trajectory shows more irregular movements due to stochastic gradient estimates, whereas NAG introduces momentum that can accelerate movement toward the optimum. Adagrad adapts the learning rate for each parameter individually, which leads to a more stable trajectory and allows it to reach a solution very close to the optimal parameters.

Q2:

Synthetic data for the nonlinear model:

$$\hat{y}(u;\, x) = x_2 tanh(x_1 u), \qquad x = [x_1, x_2]^T,$$

was generated in accordance with the assignment's specifications. A dataset of size $m = 1000$ was created where the input variable $u^{(i)}$ was sampled from the uniform distribution on the interval $[-2,2]$. The true parameter vector was set to $x^* = [1, 3]^T$, and the corresponding target values were generated using the model with an additive noise term $\epsilon^{(i)} \sim N(0, 0.05^2)$.

```
m = 1000
x_star = np.array([1,3])
u = np.random.uniform(-2,2,m)
epsilon = 0.05*np.random.randn(m)
```

The loss function used was the mean squared error loss

$$J(x) = \frac{1}{2m}\sum_{i=1}^{m}\left(\hat{y}(u^{(i)};\, x) - y^{(i)}\right)^2 \quad \text{where} \quad y^{(i)} = \hat{y}(u^{(i)};\, x^*) + \epsilon^{(i)}$$

The parameters were initialised at $x_0 = [1, 1]^T$.

```
def model(u,x):
    return x[1]*np.tanh(x[0]*u)

y = model(u,x_star) + epsilon
x0 = np.array([1.,1.])

def loss(x):
    yhat = model(u,x)
    return (1/(2*m))*np.sum((yhat-y)**2)

def grad(x):
    yhat = model(u,x)
    e = yhat - y
    tanh_term = np.tanh(x[0]*u)
    g2 = (1/m)*np.sum(e * tanh_term)
    g1 = (1/m)*np.sum(e * x[1] * (1 - tanh_term**2) * u)
    return np.array([g1,g2])
```

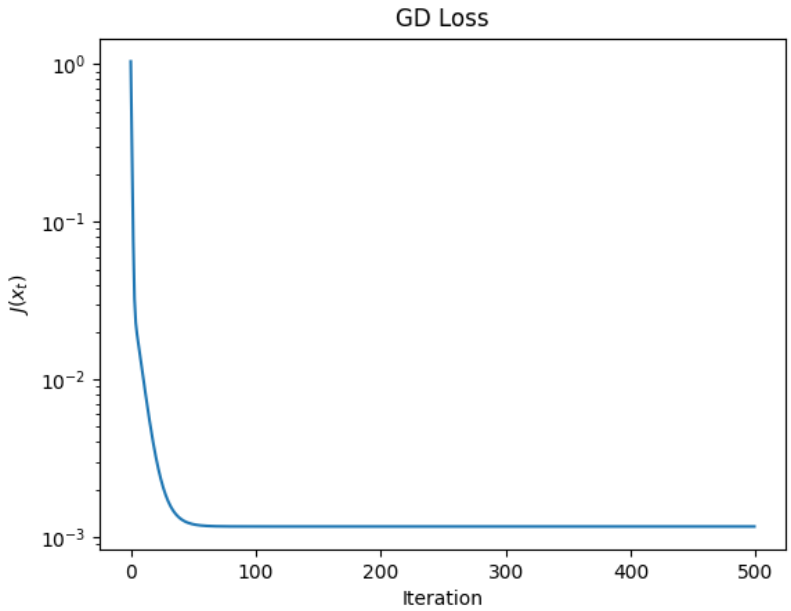
In the implementation, a function was defined for the model $x_2 tanh(x_1 u)$, along with functions computing the loss and its gradient. The gradient expressions (`g1` and `g2`) were implemented using the derivative of the hyperbolic tangent function and implemented using vectorised NumPy operations.

(a) Full-batch Gradient Descent

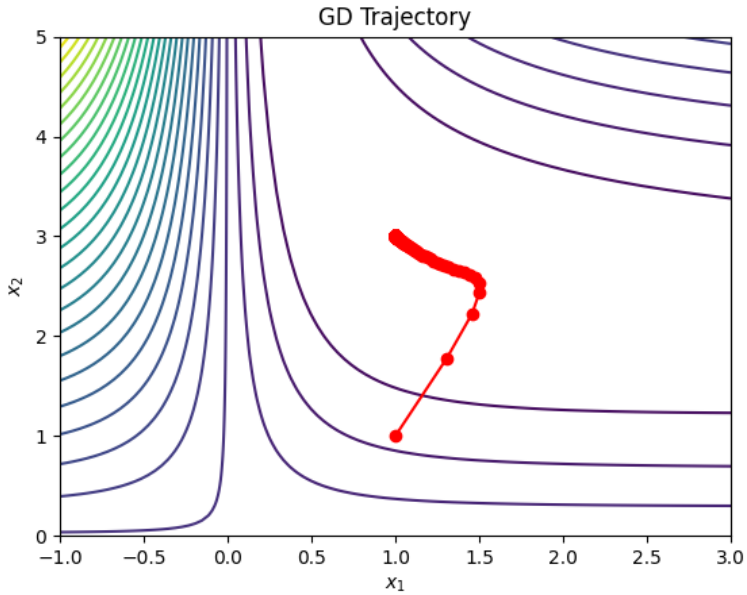
As a baseline, full-batch gradient descent was implemented. At each iteration the gradient of the loss was computed using the entire dataset and the parameters were updated using the rule $x_{t+1} = x_t - \alpha \nabla J(x_t)$, with a constant step size $\alpha = 0.75$. The algorithm was run for 500 iterations starting from the initial parameter vector.

```
alpha = 0.75
T = 500
x = x0.copy()

losses_gd = []
traj_gd = [x.copy()]

for t in range(T):
    losses_gd.append(loss(x))
    x = x - alpha*grad(x)
    traj_gd.append(x.copy())

traj_gd = np.array(traj_gd)
```

During optimisation, the value of the loss and trajectory at each iteration were stored to allow visualisation of the optimisation process.



The first plot shows the objective value $J(x_t)$ against the iteration number on a logarithmic scale. The loss decreases rapidly during optimisation, falling from approximately 1.03 to around 1.23×10^{-3} by the final iteration. It is apparent that gradient descent can effectively minimise the objective function despite the nonlinear structure of the model. The parameters converge to an estimate of approximately $x = (1.005,\ 2.991)$, which is close to the true parameter values $(1, 3)$, with a parameter error of approximately 0.011.



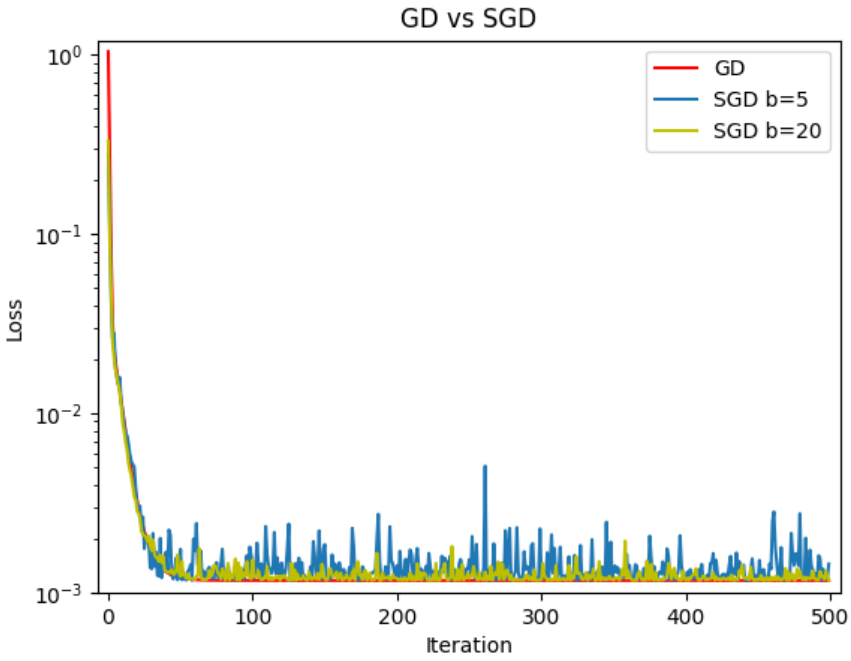
The second plot shows the optimisation trajectory on a contour plot of the objective function $J(x_1, x_2)$. The contour surface was constructed by evaluating the loss over a grid of parameter values, and the sequence of parameter updates was plotted on top of this surface to visualise how gradient descent moves through the parameter space. The trajectory demonstrates how the algorithm gradually approaches the optimal region corresponding to the true parameters.

(b) Mini-batch Stochastic Gradient Descent

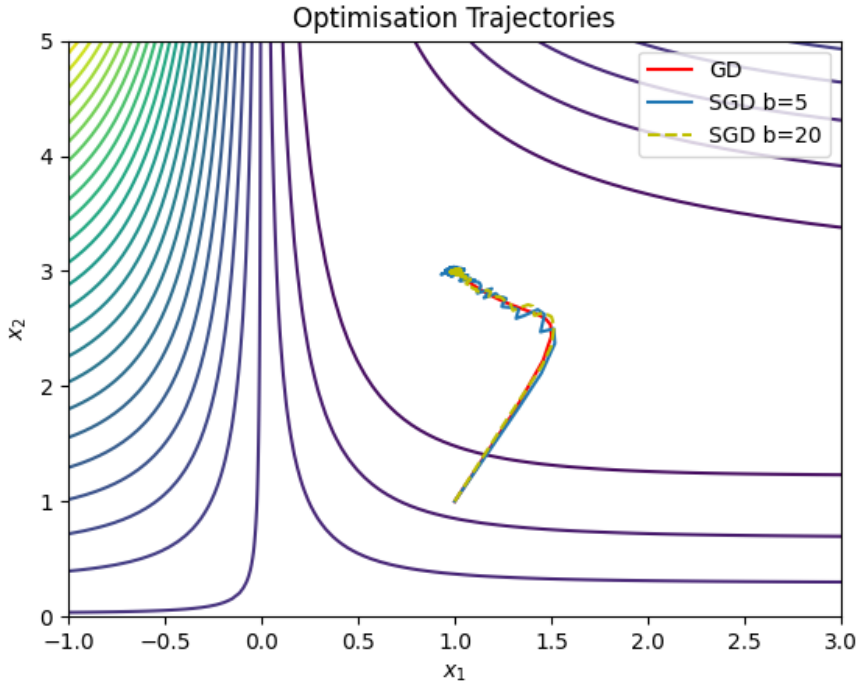
Mini-batch stochastic gradient descent (SGD) was implemented to investigate the effect of stochastic gradient estimates on optimisation behaviour. Instead of using the full dataset as was done in Q1(b), gradients were computed using small randomly selected mini batches of the data.

```
def SGD(batch_size, updates, alpha):
    x = x0.copy()
    traj = [x.copy()]
    losses = []
    idx = np.arange(m)

    for t in range(updates):
        if t % (m//batch_size) == 0:
            np.random.shuffle(idx)

        batch = idx[(t*batch_size)%m : (t*batch_size)%m + batch_size]
        ub = u[batch]
        yb = y[batch]

        yhat = x[1]*np.tanh(x[0]*ub)
        e = yhat - yb
        tanh_term = np.tanh(x[0]*ub)

        g2 = (1/batch_size)*np.sum(e*tanh_term)
        g1 = (1/batch_size)*np.sum(e*x[1]*(1-tanh_term**2)*ub)
        g = np.array([g1,g2])
        x = x - alpha*g

        losses.append(loss(x))
        traj.append(x.copy())

    return np.array(losses), np.array(traj)
```

The dataset indices were shuffled once per epoch to ensure randomness in batch selection. The algorithm was run for 500 parameter updates using the same learning rate $\alpha = 0.75$. Two batch sizes were compared: $b = 5$ and $b = 20$. The loss values and trajectories of SGD for each batch size were stored and plotted against the number of iterations.



The first plot compares the loss curves of full-batch GD and the two SGD variants. All three methods successfully minimise the objective function to very small values (around 10^{-3}). The SGD curves exhibit small fluctuations due to the stochastic nature of the mini-batch gradient estimates. In this experiment, the final losses are approximately 0.00126 for $b = 5$ and 0.00143 for $b = 20$, both close to the GD result of 0.00123. Although the smaller batch size produces noisier updates, both batch sizes still converge to parameter estimates close to the true values.



The second plot displays the optimisation trajectories for GD and both SGD variants on the same contour plot of the objective function. The GD trajectory appears smooth and follows a direct path toward the minimum. In contrast, the SGD trajectories exhibit small irregularities caused by the randomness of the mini-batch gradients. However, all trajectories ultimately converge toward the same optimal region near the true parameters.

(c) <u>SGD Noise: NAG and Adagrad</u>

To further analyse the behaviour of stochastic optimisation methods, two extensions of SGD were implemented: Nesterov Accelerated Gradient (NAG) and Adagrad. Both methods were implemented as functions using a mini-batch size of $b = 10$, in the same manner as in Q1 though accounting for `x` instead of `theta` and the nonlinear model with its gradient expressions (`g1` and `g2`).

```
def NAG(batch_size,updates,alpha,beta=0.9):
    x = x0.copy()
    v = np.zeros_like(x)
    traj=[x.copy()]
    losses=[]
    idx=np.arange(m)
```

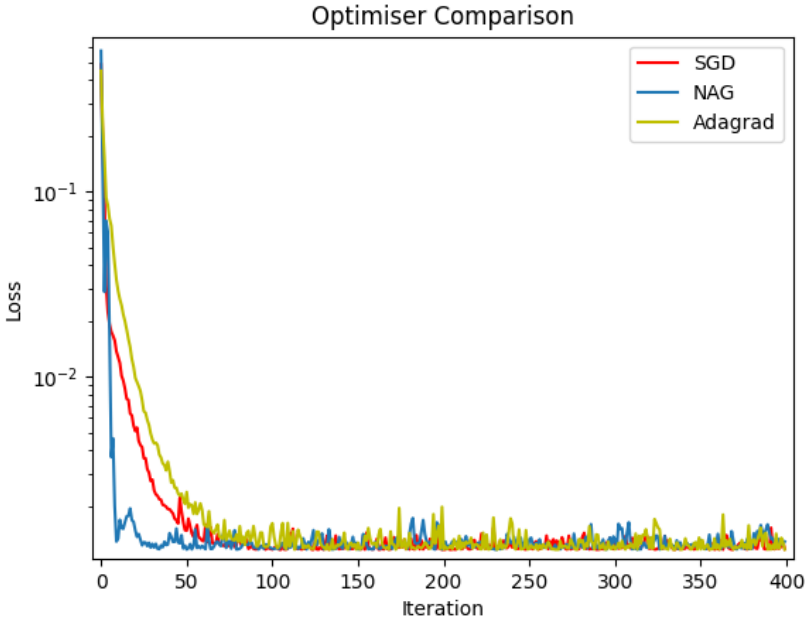
```
    for t in range(updates):
        if t%(m//batch_size)==0:
            np.random.shuffle(idx)

        batch=idx[(t*batch_size)%m:(t*batch_size)%m+batch_size]
        ub=u[batch]
        yb=y[batch]

        yhat = lookahead[1]*np.tanh(lookahead[0]*ub)
        e = yhat - yb
        tanh_term = np.tanh(lookahead[0]*ub)

        g2 = (1/batch_size)*np.sum(e*tanh_term)
        g1 = (1/batch_size)*np.sum(e*lookahead[1]*(1-tanh_term**2)*ub)
        g=np.array([g1,g2])

        v = beta*v + alpha*g
        x = x - v
        losses.append(loss(x))
        traj.append(x.copy())

    return np.array(losses), np.array(traj)
```

NAG incorporates momentum by maintaining a velocity vector that accumulates previous gradients and evaluates the gradient at a look-ahead position to improve convergence speed. A momentum coefficient $\beta = 0.9$ was used in the implementation.

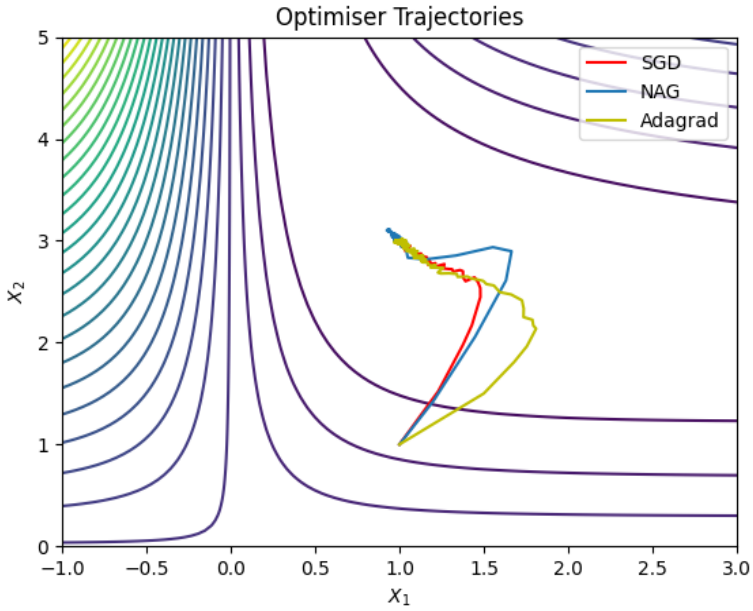
```
def Adagrad(batch_size,updates,alpha,eps=1e-8):
    x = x0.copy()
    G = np.zeros_like(x)
    traj=[x.copy()]
    losses=[]
    idx=np.arange(m)

    for t in range(updates):
        if t%(m//batch_size)==0:
            np.random.shuffle(idx)

        batch=idx[(t*batch_size)%m:(t*batch_size)%m+batch_size]
        ub=u[batch]
        yb=y[batch]

        yhat = x[1]*np.tanh(x[0]*ub)
        e = yhat - yb
        tanh_term = np.tanh(x[0]*ub)

        g2 = (1/batch_size)*np.sum(e*tanh_term)
        g1 = (1/batch_size)*np.sum(e*x[1]*(1-tanh_term**2)*ub)
        g=np.array([g1,g2])
        G += g**2

        x = x - alpha/(np.sqrt(G)+eps)*g
        losses.append(loss(x))
        traj.append(x.copy())

    return np.array(losses), np.array(traj)
```

Adagrad, on the other hand, adapts the learning rate for each parameter individually by accumulating the squared gradients over time and scaling updates accordingly.

The loss values and trajectories of the optimisation methods were stored and plotted against the number of iterations.



The first plot compares the loss curves for standard SGD, NAG, and Adagrad. All three methods converge to very small loss values on the order of 10^{-3}. The final losses are approximately 0.00124 for SGD, 0.00130 for NAG, and 0.00128 for Adagrad, indicating that all methods perform similarly well on this problem. Each optimiser converges to parameter estimates that are very close to the true parameters $(1, 3)$.



The second plot shows the trajectories of the three methods on the contour plot of $J(x_1, x_2)$. Although the paths taken by the algorithms differ slightly due to their update rules, all three methods move toward the same optimal region in parameter space. The trajectories illustrate how different optimisation strategies can reach similar solutions while following different paths during optimisation.

Q3:

The optimisation algorithms were applied to the Rosenbrock function, a standard benchmark function used to test optimisation methods. The function is defined as:

$$f(x_1, x_2) = (1 - x_1)^2 + 100(x_2 - x_1^2)^2,$$

which has a global minimum at (1,1). The optimisation was initialised at $x_0 = [-1, 1]^T$.

```
def f(x):
    x1, x2 = x
    return (1-x1)**2 + 100*(x2 - x1**2)**2

x0 = np.array([-1.,1.])

def grad_fd(x,h=1e-5):
    g = np.zeros(2)
    for i in range(2):
        xp = x.copy()
        xm = x.copy()

        xp[i] += h
        xm[i] -= h

        g[i] = (f(xp)-f(xm))/(2*h)

    return g


def hessian_fd(x,h=1e-4):
    H = np.zeros((2,2))
    for i in range(2):
        for j in range(2):
            xp = x.copy()
            xm = x.copy()
            xpp = x.copy()
            xmm = x.copy()

            xp[i]+=h; xp[j]+=h
            xm[i]+=h; xm[j]-=h
            xpp[i]-=h; xpp[j]+=h
            xmm[i]-=h; xmm[j]-=h

            H[i,j]=(f(xp)-f(xm)-f(xpp)+f(xmm))/(4*h*h)

    return H
```

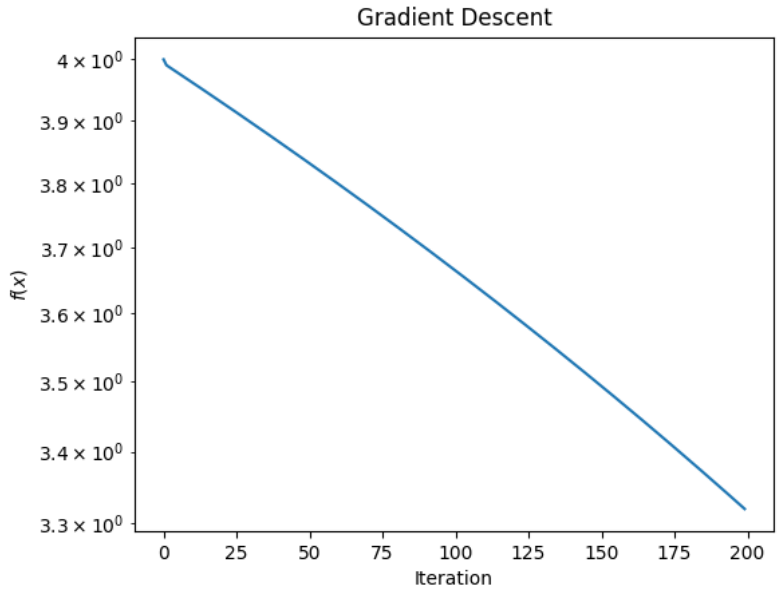
In the implementation, a function was defined to evaluate $f(x)$. Additionally functions to compute the gradient and Hessian using finite difference approximations were implemented. The gradient was estimated using a central difference formula applied separately to each coordinate, while the Hessian matrix was computed using second-order finite differences.

(a) Gradient Descent Baseline

As a baseline method, gradient descent was implemented. At each iteration the gradient was evaluated using the finite difference function, and the parameters were updated according to $x_{t+1} = x_t - \alpha \nabla f(x_t)$ with a constant step size $\alpha = 10^{-3}$. The algorithm was run for 200 iterations, and both the objective value and parameter vector were recorded at each step.

```
alpha = 1e-3
T = 200
x = x0.copy()

losses_gd = []
traj_gd = [x.copy()]

for t in range(T):
    loss_gd.append(f(x))
    g = grad_fd(x)
    x = x - alpha*g
    traj_gd.append(x.copy())

traj_gd = np.array(traj_gd)
```

During optimisation, the value of the loss and trajectory at each iteration were stored and used to plot the value of $f(x_t)$ versus iteration on a logarithmic scale.



This plot illustrates the convergence behaviour of gradient descent when applied to the Rosenbrock function, which is known for its narrow-curved valley. Starting from the initial point $x_0 = (-1, 1)$, the objective value decreases only modestly from 4.0 to approximately 3.32 after 200 iterations. The final parameter estimates $(-0.82, 0.68)$ remains far from the true minimum at $(1, 1)$ with approximately 1.85 from the optimum. This slow progress reflects the difficulty that gradient descent encounters when navigating the curved valley of the Rosenbrock function, where the gradient direction does not point directly toward the minimum.

(b) Newton Method with Finite Difference Hessian

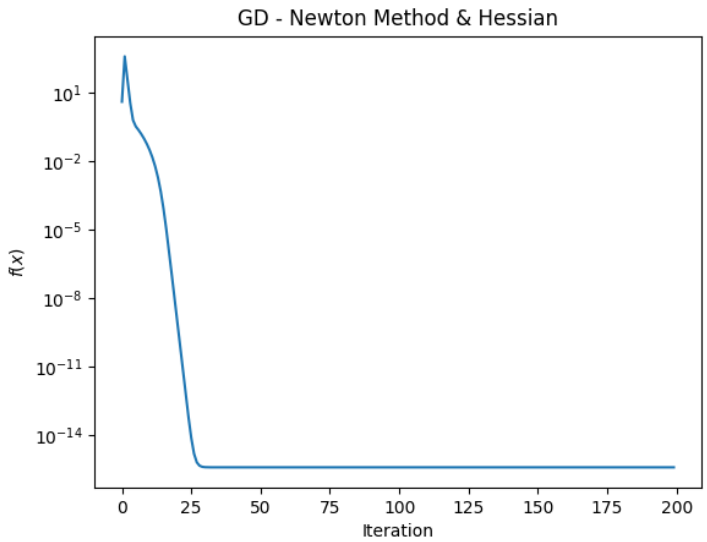
A Newton optimisation method was implemented using both the finite difference gradient and Hessian.

```
alpha = 0.7
T = 200
x = x0.copy()

loss_newton=[]
traj_newton=[x.copy()]

for t in range(T):
    loss_newton.append(f(x))
    g = grad_fd(x)
    H = hessian_fd(x)
    p = np.linalg.solve(H,g)
    x = x - alpha*p
    traj_newton.append(x.copy())

traj_newton=np.array(traj_newton)
```

At each iteration the gradient vector and Hessian matrix were computed numerically. The Newton direction was obtained by solving the linear system $Hp = g$, where H is the Hessian matrix and g is the gradient. The parameter update was then performed using

$$x_{t+1} = x_t - \alpha p$$

with a constant scaling factor $\alpha = 0.7$. The algorithm was again run for 200 iterations starting from the same initial point.



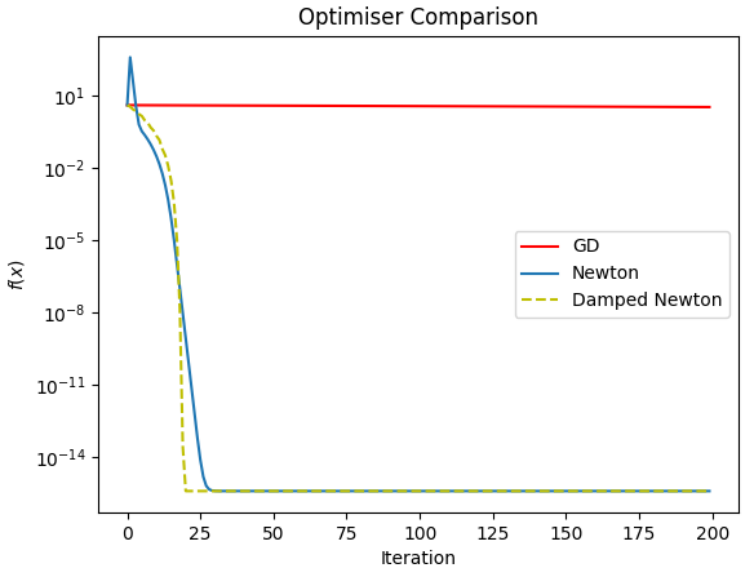
The resulting plot shows $f(x_t)$ versus iteration on a logarithmic scale. Compared with gradient descent, the Newton method reduces the objective value dramatically, reaching approximately 4×10^{-16}, which is effectively zero. The final parameter estimate is approximately $(0.99999998, 0.99999996)$, exceedingly close to the true optimum (1, 1) with a distance of approximately 4.5×10^{-8}. This rapid convergence occurs because Newton's method incorporates second-order curvature information through the Hessian matrix, allowing the algorithm to adapt its update direction to the local geometry of the objective function and move efficiently along the curved valley.

(c) Damped Newton Method

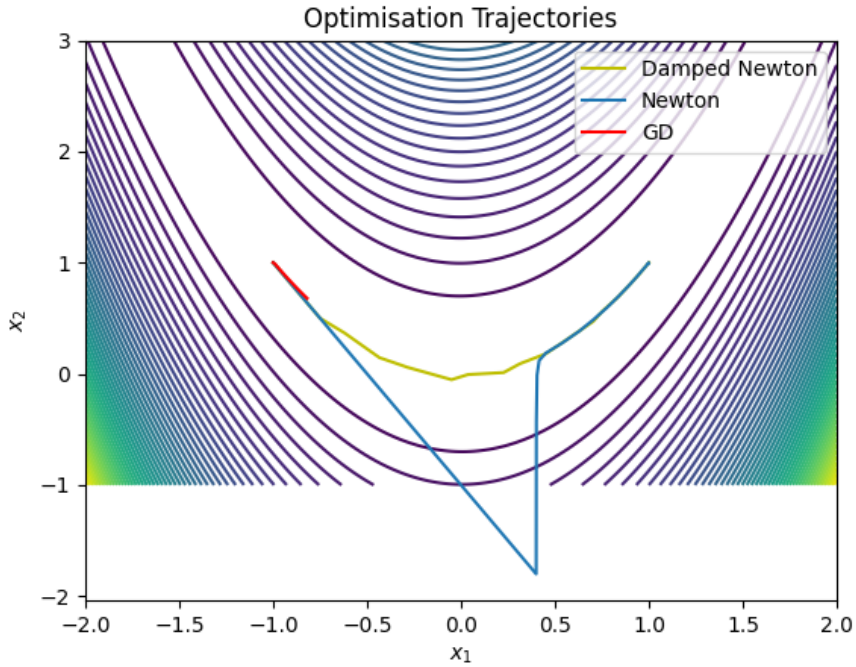
To improve stability, a damped Newton method with backtracking was also implemented. In this approach the Newton direction was computed in the same way as before, but the step size α was made to dynamically adjust itself to ensure that $f(x)$ decreased at each iteration.

```
T=200
alpha0=1
rho=0.5
K=20
x=x0.copy()

loss_dn=[]
traj_dn=[x.copy()]

for t in range(T):
    ft=f(x)
    loss_dn.append(ft)
    g=grad_fd(x)
    H=hessian_fd(x)
    p=np.linalg.solve(H,g)
    alpha=alpha0
    accepted=False
    for k in range(K):
        x_new=x-alpha*p
        if f(x_new)<ft:
            accepted=True
            break

        alpha*=rho
    if not accepted:
        x_new=x-1e-4*g
    x=x_new
    traj_dn.append(x.copy())

traj_dn=np.array(traj_dn)
```

Starting from an initial step size $\alpha_0 = 1$, the step size was repeatedly reduced by a shrink factor $\rho = 0.5$ until a decrease in the objective value was achieved or a maximum number of backtracking steps was reached. If no acceptable step was found, a small gradient descent step was used as a fallback.



The first plot compares the objective values $f(x_t)$ for gradient descent, the standard Newton method, and the damped Newton method on a logarithmic scale. The results highlight the substantial difference in convergence speed between first- and second-order methods. While gradient descent shows only a small reduction in the objective value, both Newton and damped Newton rapidly reduce the loss to values on the order of 10^{-16}



The second plot shows contour lines of the Rosenbrock function together with the optimisation trajectories of the three methods. The gradient descent trajectory progresses slowly along the curved valley and remains far from the optimum within the given number of iterations. In contrast, both Newton and damped Newton promptly move toward the global minimum at $(1, 1)$. The trajectories illustrate how second-order methods can exploit curvature information to follow the valley more effectively than gradient descent.